

Session 3 _ ossp_2520090235

Task1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

rohita@LaxmiDurga: ~

```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>

#define MAX_BUFFER 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_BUFFER];
int history_count = 0;
int history_index = 0;

void enable_raw_mode(struct termios *old) {
    struct termios raw;

    tcgetattr(STDIN_FILENO, old);
    raw = *old;

    raw.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void restore_mode(struct termios *old) {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, old);
}

void save_command(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 1; i < MAX_HISTORY; i++)
            strcpy(history[i - 1], history[i]);

        strcpy(history[MAX_HISTORY - 1], command);
    }

    history_index = history_count;
}

int main() {
    struct termios old;
    char buffer[MAX_BUFFER];
    int index = 0;
    int ch;

    strcpy(history[0], "uname -a");
    strcpy(history[1], "lscpu");
    strcpy(history[2], "ps aux");
    history_count = 3;
    history_index = history_count;

    printf("Session 3 - Command History\n");
    printf("Type commands. Use UP/DOWN arrows for history.\n");
    printf("Press Ctrl+C to stop.\n\n");

    enable_raw_mode(&old);
```



```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
#define INITIAL_SIZE 10  
  
typedef struct Node {  
    char *command;  
    struct Node *next;  
} Node;  
  
int main() {  
    char *buffer;  
    size_t size = INITIAL_SIZE;  
    size_t length = 0;  
    int ch;  
  
    Node *head = NULL;  
    Node *tail = NULL;  
  
    buffer = malloc(size);  
  
    if (buffer == NULL) {  
        perror("malloc failed");  
        return 1;  
    }  
  
    printf("Session 3 - Dynamic Memory Management\n");  
    printf("Enter commands. Type 'exit' to finish.\n\n");  
  
    while (1) {  
        length = 0;  
  
        printf("myshell> ");  
        fflush(stdout);  
  
        while ((ch = getchar()) != '\n' && ch != EOF) {  
            if (length + 1 >= size) {  
                size *= 2;  
  
                char *temp = realloc(buffer, size);  
  
                if (temp == NULL) {  
                    perror("realloc failed");  
                    free(buffer);  
                    return 1;  
                }  
  
                buffer = temp;  
            }  
  
            buffer[length++] = ch;  
  
            buffer[length] = '\0';  
  
            if (strcmp(buffer, "exit") == 0) {  
                break;  
            }  
  
            if (length == 0) {  
                continue;  
            }  
  
            Node *new_node = malloc(sizeof(Node));  
  
            if (new_node == NULL) {  
                perror("malloc failed");  
                break;  
            }  
  
            new_node->command = malloc(length + 1);  
  
            if (new_node->command == NULL) {  
                perror("malloc failed");  
                free(new_node);  
                break;  
            }  
  
            strcpy(new_node->command, buffer);  
            new_node->next = NULL;  
  
            if (head == NULL) {  
                head = new_node;  
                tail = new_node;  
            } else {  
                tail->next = new_node;  
                tail = new_node;  
            }  
  
            printf("Stored command: %s\n", buffer);  
            printf("Buffer size: %zu bytes\n", size);  
  
            free(buffer);  
  
            Node *current = head;  
  
            while (current != NULL) {  
                Node *next = current->next;  
  
                free(current->command);  
                free(current);  
  
                current = next;  
            }  
  
            printf("\nAll dynamically allocated memory released.\n");  
            return 0;  
        }  
    }  
}
```

Output

```
rohita@LaxmiDurga:~$ ./a.out  
Session 3 - Dynamic Memory Management  
Enter commands. Type 'exit' to finish.  
  
myshell> Hello, Good morning  
Stored command: Hello, Good morning  
Buffer size: 20 bytes  
myshell> _
```

Session 4

Task1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50
#define MAX_INPUT 500

typedef struct {
    char value[MAX_TOKEN_LENGTH];
    char type[20];
} Token;

int main() {
    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("Task 1: Tokenization Engine & Stream Validator\n");
    printf("Enter a command stream (e.g., 'cat file.txt | grep error ; exit'):\n");
    printf("mysHELL ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 0;
    }

    int i = 0;
    while (input[i] != '\0' && input[i] != '\n') {
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            break;
        }

        if (token_count >= MAX_TOKENS) {
            printf("\n[DEBUG ERROR] Parse aborted: Exceeded maximum token capacity (%d).\n", MAX_TOKENS);
            break;
        }

        int j = 0;

        if (input[i] == ';' || input[i] == '|') {
            tokens[token_count].value[0] = input[i];
            tokens[token_count].value[1] = '\0';
            strcpy(tokens[token_count].type, "DELIMITER");
            token_count++;
            i++;
        }
        else {
            while (input[i] != '\0' &&
                input[i] != '\n' &&
                !isspace((unsigned char)input[i]) &&
                input[i] != ';' &&
                input[i] != '|') {

                if (j < MAX_TOKEN_LENGTH - 1) {
                    tokens[token_count].value[j++] = input[i];
                }
            }
            tokens[token_count].value[j] = '\0';
            strcpy(tokens[token_count].type, "COMMAND/ARG");
            token_count++;
            i++;
        }
    }
}
```

```

        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count].value[j++] = input[i];
        }
        i++;
    }
    tokens[token_count].value[j] = '\0';
    strcpy(tokens[token_count].type, "COMMAND/ARG");

    if (j > 0) {
        token_count++;
    }
}

printf("===== PARSER DEBUG OUTPUT =====\n");
printf("Total Tokens Identified: %d\n", token_count);
for (int k = 0; k < token_count; k++) {
    printf("Token [%02d] -> Type: %12s | Value: [%s]\n", k + 1, tokens[k].type, tokens[k].value);
}
printf("===== \n");

printf("\nToken Stream Validation Status: ");
int valid = 1;

if (token_count == 0) {
    printf("INVALID (Empty command stream provided).\n");
    return 0;
}

if (strcmp(tokens[0].type, "DELIMITER") == 0) {
    valid = 0;
    printf("INVALID\n -> Error: Stream cannot start with a delimiter '%s'.\n", tokens[0].value);
}
else {
    for (int k = 0; k < token_count; k++) {
        if (strcmp(tokens[k].type, "DELIMITER") == 0) {
            if (k == token_count - 1 && tokens[k].value[0] == '|') {
                valid = 0;
                printf("INVALID\n -> Error: Dangling pipe operator '%s' at end of stream.\n", tokens[k].value);
                break;
            }
            if (k < token_count - 1 && strcmp(tokens[k + 1].type, "DELIMITER") == 0) {
                valid = 0;
                printf("INVALID\n -> Error: Consecutive delimiters detected around '%s %s'.\n", tokens[k].value, tokens[k + 1].value);
                break;
            }
        }
    }
}

if (valid) {
    printf("SUCCESSFULLY VALIDATED.\n");
}

return 0;
}
```

```

GNU nano 8.7.1                                OS.C
    input[i] != '\n' &&
    !isspace((unsigned char)input[i]) &&
    input[i] != ';' &&
    input[i] != '|' ) {

        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count].value[j++] = input[i];
        }
        i++;
    }
    tokens[token_count].value[j] = '\0';
    strcpy(tokens[token_count].type, "COMMAND/ARG");

    if (j > 0) {
        token_count++;
    }
}

printf("\n===== PARSER DEBUG OUTPUT =====\n");
printf("Total Tokens Identified: %d\n", token_count);
for (int k = 0; k < token_count; k++) {
    printf("Token [%02d] -> Type: %-12s | Value: [%s]\n", k + 1, tokens[k].type, tokens[k].value);
}
printf("===== \n");

printf("\nToken Stream Validation Status: ");
int valid = 1;

if (token_count == 0) {
    printf("INVALID (Empty command stream provided).\n");
    return 0;
}

if (strcmp(tokens[0].type, "DELIMITER") == 0) {
    valid = 0;
    printf("INVALID\n -> Error: Stream cannot start with a delimiter '%s'.\n", tokens[0].value);
}
else {
    for (int k = 0; k < token_count; k++) {
        if (strcmp(tokens[k].type, "DELIMITER") == 0) {
            if (k == token_count - 1 && tokens[k].value[0] == '|') {
                valid = 0;
                printf("INVALID\n -> Error: Dangling pipe operator '%s' at end of stream.\n", tokens[k].value);
                break;
            }
            if (k < token_count - 1 && strcmp(tokens[k + 1].type, "DELIMITER") == 0) {
                valid = 0;
                printf("INVALID\n -> Error: Consecutive delimiters detected around '%s %s'.\n", tokens[k].value, tokens[k + 1].value);
                break;
            }
        }
    }
}

if (valid) {
    printf("SUCCESSFULLY VALIDATED.\n");
}

return 0;
}

```

Output:

```

rohita@LaxmiDurga:~$ gcc OS.C
rohita@LaxmiDurga:~$ ./a.out
Task 1: Tokenization Engine & Stream Validator
Enter a command stream (e.g., 'cat file.txt | grep error ; exit'):
myshell> 'cat file.txt | grep error ; exit'

===== PARSER DEBUG OUTPUT =====
Total Tokens Identified: 7
Token [01] -> Type: COMMAND/ARG | Value: ['cat]
Token [02] -> Type: COMMAND/ARG | Value: [file.txt]
Token [03] -> Type: DELIMITER | Value: [|]
Token [04] -> Type: COMMAND/ARG | Value: [grep]
Token [05] -> Type: COMMAND/ARG | Value: [error]
Token [06] -> Type: DELIMITER | Value: [;]
Token [07] -> Type: COMMAND/ARG | Value: [exit']
=====

```

Task 3:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

int main() {
    char input[500];
    Token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("Session 4 - Tokenization\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    int i = 0;
    while (input[i] != '\0' && input[i] != '\n') {

        /* Skip whitespace */
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n') {
            break;
        }

        /* Check token limit */
        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        int j = 0;

        /* Read characters until whitespace or delimiter */
        while (input[i] != '\0' &&
            input[i] != '\n' &&
            !isspace((unsigned char)input[i]) &&
            input[i] != ';' &&
            input[i] != '|') {

            if (j < MAX_TOKEN_LENGTH - 1) {
                tokens[token_count].value[j++] = input[i];
            }
            i++;
        }

        tokens[token_count].value[j] = '\0';

        if (j > 0) {
            token_count++;
        }

        /* Handle delimiters */
        if (input[i] == ';' || input[i] == '|') {

```

```

GNU nano 8.7.1
    input[i] != '\n' &&
    !isspace((unsigned char)input[i]) &&
    input[i] != ';' &&
    input[i] != '|' {
        if (j < MAX_TOKEN_LENGTH - 1) {
            tokens[token_count].value[j++] = input[i];
        }
        i++;
    }
    tokens[token_count].value[j] = '\0';
    if (j > 0) {
        token_count++;
    }
    /* Handle delimiters */
    if (input[i] == ';' || input[i] == '|') {
        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }
        tokens[token_count].value[0] = input[i];
        tokens[token_count].value[1] = '\0';
        token_count++;
        i++;
    }
}
printf("\nTokens identified:\n");
if (token_count == 0) {
    printf("No tokens found.\n");
    return 0;
}
for (int k = 0; k < token_count; k++) {
    printf("Token %d: [%s]\n", k + 1, tokens[k].value);
}
printf("\nToken stream validation:\n");
int valid = 1;
for (int k = 0; k < token_count; k++) {
    if (strlen(tokens[k].value) == 0) {
        valid = 0;
    }
}
if (valid)
    printf("Token stream is valid.\n");
else
    printf("Token stream is invalid.\n");
return 0;

```

Output:

```

Session 4 - Tokenization
Enter a command: ls -l | grep txt

Tokens identified:
Token 1: [ls]
Token 2: [-l]
Token 3: [|]
Token 4: [grep]
Token 5: [txt]

Token stream validation:
Token stream is valid.

```

Task 4:

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50

int main() {
    char input[500];
    char *tokens[MAX_TOKENS];
    int token_count = 0;
    int valid = 1;

    printf("Session 4 - Parser\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    /* Tokenize input */
    char *token = strtok(input, " \t");
    while (token != NULL && token_count < MAX_TOKENS) {
        tokens[token_count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\nParse Structure:\n");

    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    /*
     * Basic syntax validation:
     * A pipe cannot be the first or last token.
     * Two pipes cannot occur consecutively.
     */
    for (int i = 0; i < token_count; i++) {
        if (strcmp(tokens[i], "|") == 0) {
            if (i == 0 || i == token_count - 1) {
                valid = 0;
                printf("Syntax Error: Invalid pipe position.\n");
            }

            if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
                valid = 0;
                printf("Syntax Error: Consecutive pipes detected.\n");
            }
        }
    }

    if (!valid) {
        printf("\nParsing failed.\n");
        return 1;
    }
}

```



```

}

/*
 * Basic syntax validation:
 * A pipe cannot be the first or last token.
 * Two pipes cannot occur consecutively.
 */
for (int i = 0; i < token_count; i++) {
    if (strcmp(tokens[i], "|") == 0) {
        if (i == 0 || i == token_count - 1) {
            valid = 0;
            printf("Syntax Error: Invalid pipe position.\n");
        }

        if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
            valid = 0;
            printf("Syntax Error: Consecutive pipes detected.\n");
        }
    }
}

if (!valid) {
    printf("\nParsing failed.\n");
    return 1;
}

/* Display execution structure */
printf("Command: %s\n", tokens[0]);

if (token_count > 1) {
    printf("Arguments:\n");

    for (int i = 1; i < token_count; i++) {
        if (strcmp(tokens[i], "|") != 0) {
            printf("  Argument %d: %s\n", i, tokens[i]);
        }
    }
}

printf("\nParse tree / execution structure:\n");
printf("ROOT\n");
printf("└─ COMMAND: %s\n", tokens[0]);

for (int i = 1; i < token_count; i++) {
    if (strcmp(tokens[i], "|") == 0) {
        printf("    └─ PIPE\n");
    } else {
        printf("    └─ ARGUMENT: %s\n", tokens[i]);
    }
}

printf("\nSyntax validation: SUCCESS\n");
printf("Execution structure generated successfully.\n");

return 0;

```

Output:

```

Session 4 - Parser
Enter a command: ls

Parse Structure:
Command: ls

Parse tree / execution structure:
ROOT
└─ COMMAND: ls

Syntax validation: SUCCESS
Execution structure generated successfully.

```

Observation: **Parser Design & Syntax Validation:** Developed core logic to process user commands, establishing robust syntax validation to catch structural errors like empty commands or consecutive pipe symbols